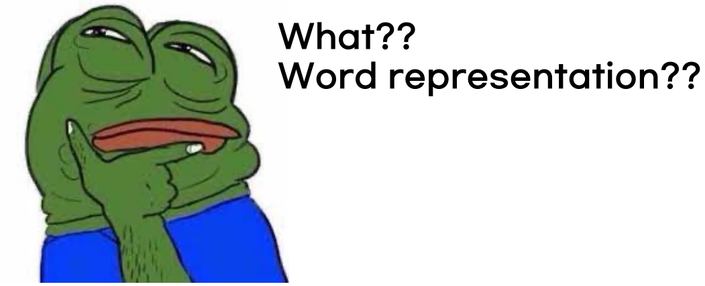
Understanding BERT from Scratch

MLP Lab

Contents

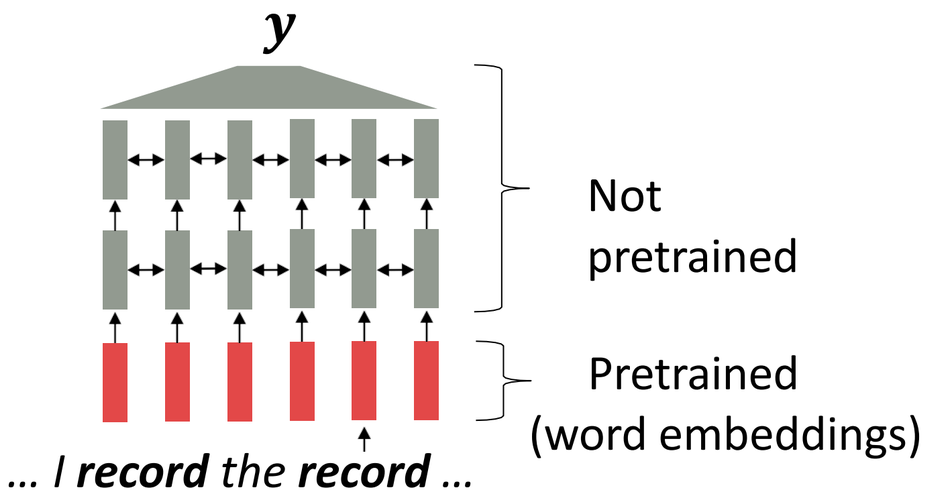
01 Introduction of BERT

- BERT (Bidirectional Encoder Representations from Transformers) is literally a bi-directional representation of a transformer encoder.
- That is, a bidirectional **word representation** using only the encoder of the transformer.

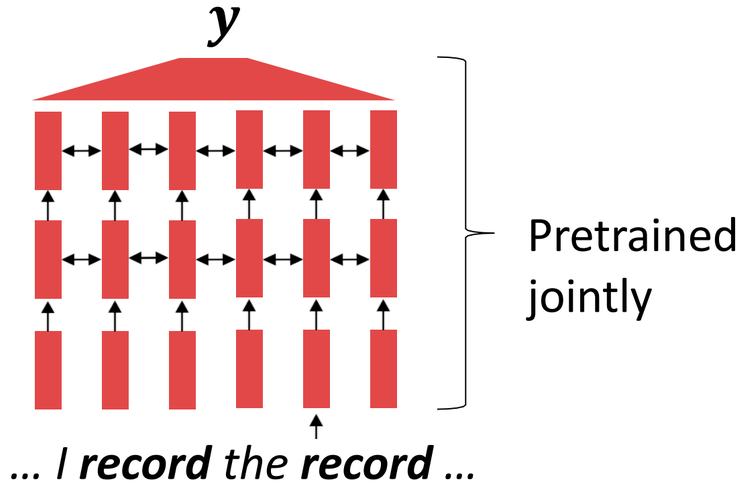


01 Introduction of BERT (Motivation)

- In order to build word embeddings as contextualized ones, our system should see the whole sentence with RNNs or Transformer.



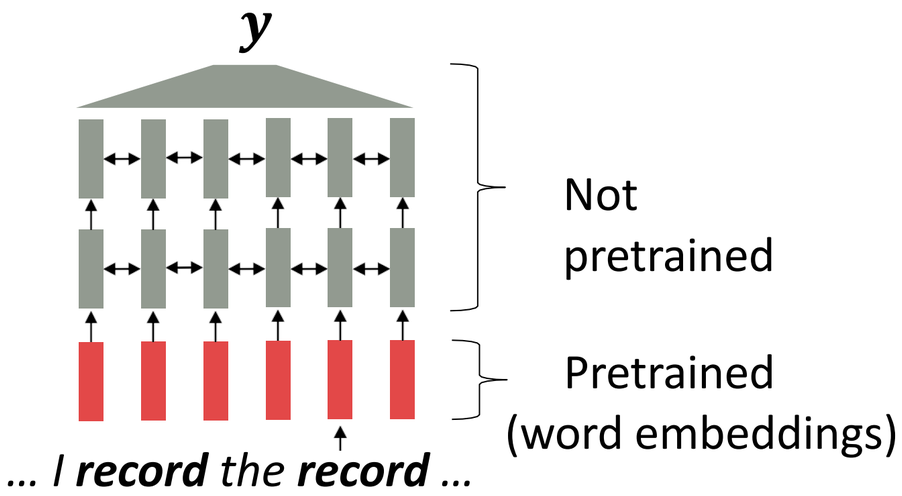
[Recall, *record* gets the same word embedding, no matter what sentence it shows up in]



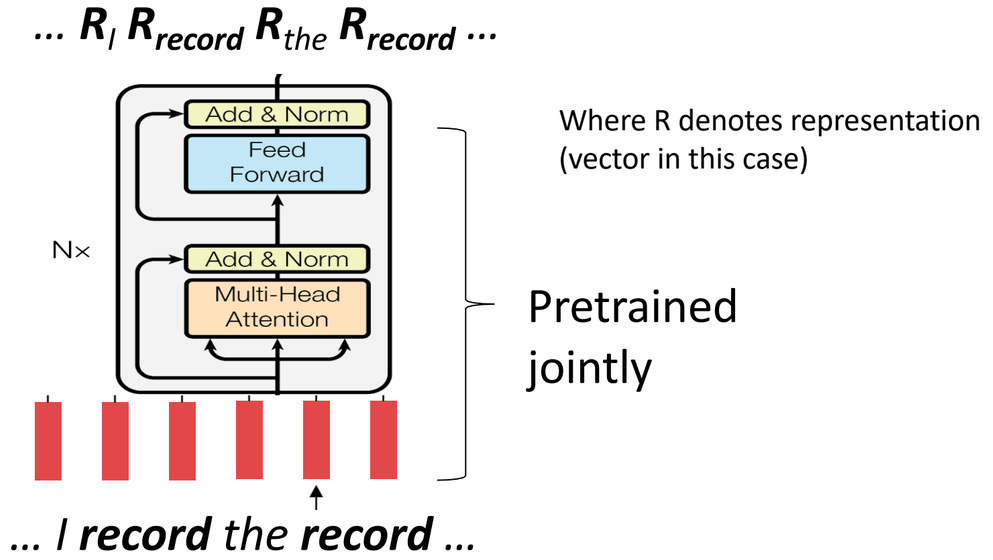
[This model has learned how to represent entire sentences through pretraining]

01 Introduction of BERT (Motivation)

- In order to build word embeddings as contextualized ones, our system should see the whole sentence with RNNs or Transformer.
- So the idea of BERT is to train word embedding with Transformer Encoder as Pretraining



[Recall, *record* gets the same word embedding, no matter what sentence it shows up in]



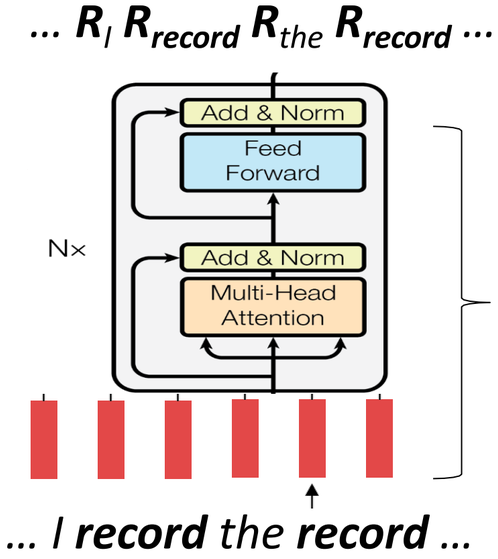
[This model has learned how to represent entire sentences through pretraining]

01 Structure of BERT

- BERT is composed of encoder layers (L), attention heads (A), and hidden layers (H).

  The standard BERT models include:

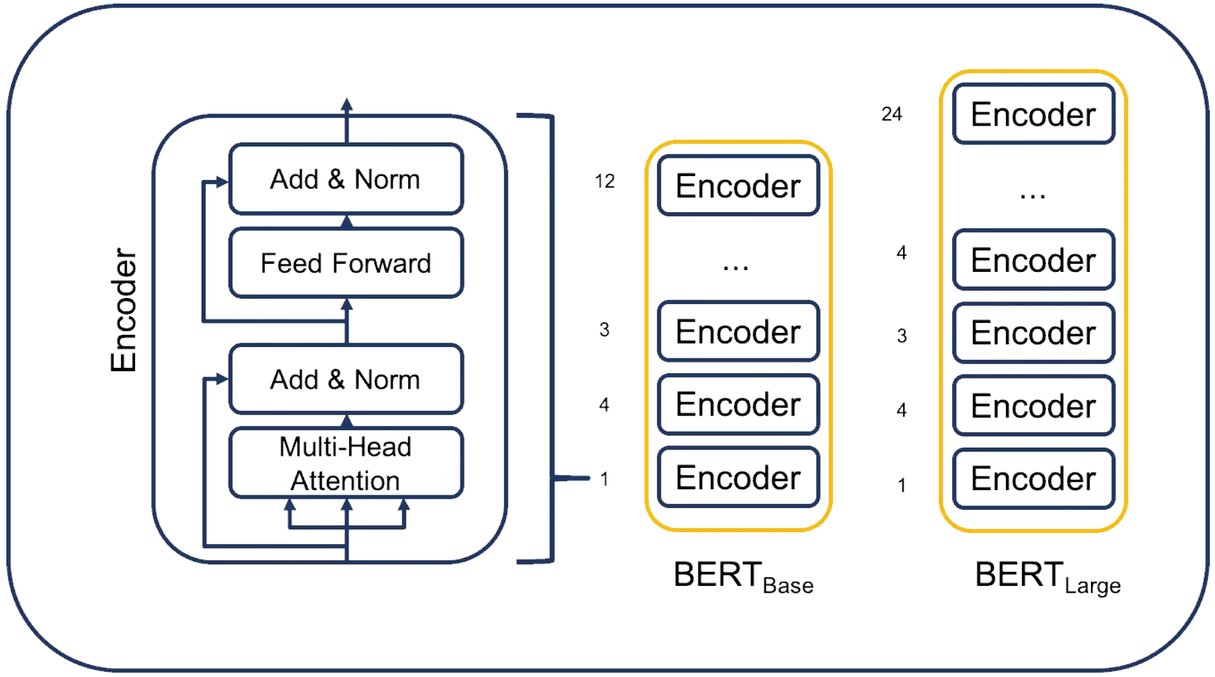
  - **BERT-base** with L=12, A=12, H=768
  - **BERT-large** with L=24, A=16, H=1024



Where R denotes representation (vector in this case)

Pretrained jointly

[This model has learned how to represent entire sentences through pretraining]

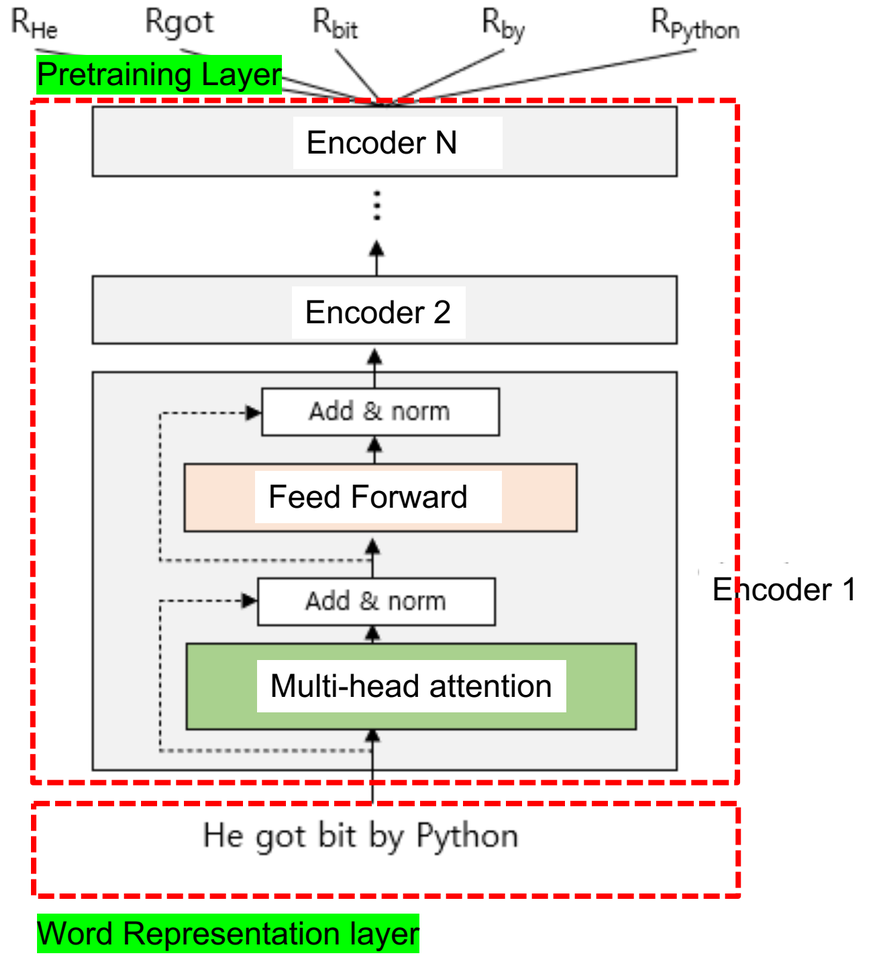


02 How BERT works

- Since the transformer's encoder can originally read sentences in both directions, BERT also reads sentences in both directions and outputs the representation.

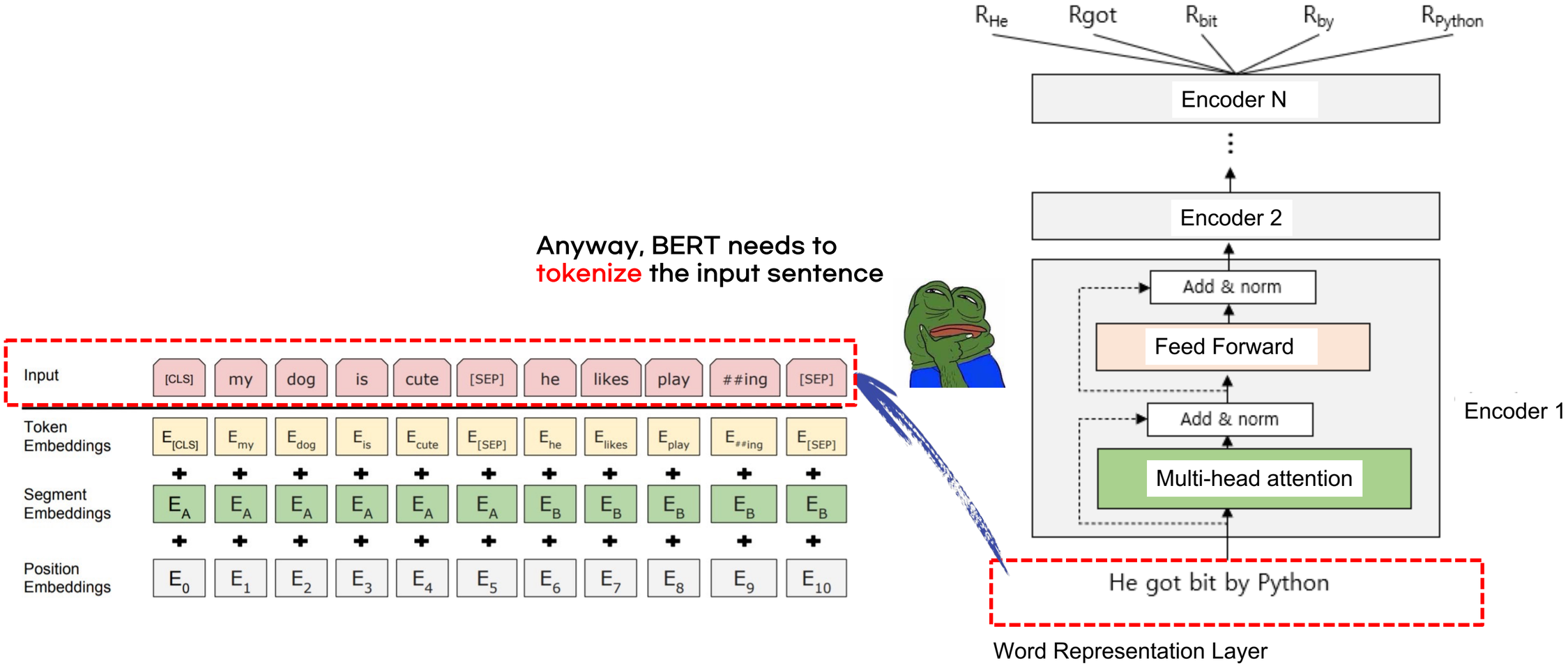
- When a sentence comes in as input to the encoder, the encoder uses a multi-head attention mechanism to connect all the words together to determine their relationships and context, and outputs a representation of each word in the sentence.

- There are two layers for training BERT
  - Word Representation Layer
  - Pretraining Layer



02 How BERT works

- Let's look deep inside of word representation layer of BERT



03 Pretraining BERT

- From word representation layer in BERT, it tokenizes the input sentence with

  (1) a sub-word tokenizer and then

  (2) transforms it into the three embeddings below, which are provided as input to BERT

  - Token embedding
  - Segment embedding
  - Position embedding

03 Pretraining BERT

- From word representation layer in BERT, it tokenizes the input sentence with

  (1) a sub-word tokenizer : There are three main subword tokenization algorithms commonly used to build vocabulary dictionaries:

  - WordPiece
  - Byte Pair Encoding (BPE)
  - Byte-level Byte Pair Encoding (BBPE)

  Among these, BERT utilizes the WordPiece algorithm.

03 Pretraining BERT

- From word representation layer in BERT, it tokenizes the input sentence with

  (1) a sub-word tokenizer : **WordPiece tokenizer**

  - WordPiece tokenizer checks to see if the word is in the lexicon, if it is, it uses the word as a token, if not, it splits the word into subwords, checking to see if the subwords are in the lexicon, and so on.

  - This method of splitting into subwords until it reaches an individual character is effective for handling out of vocabulary (OOV) words.

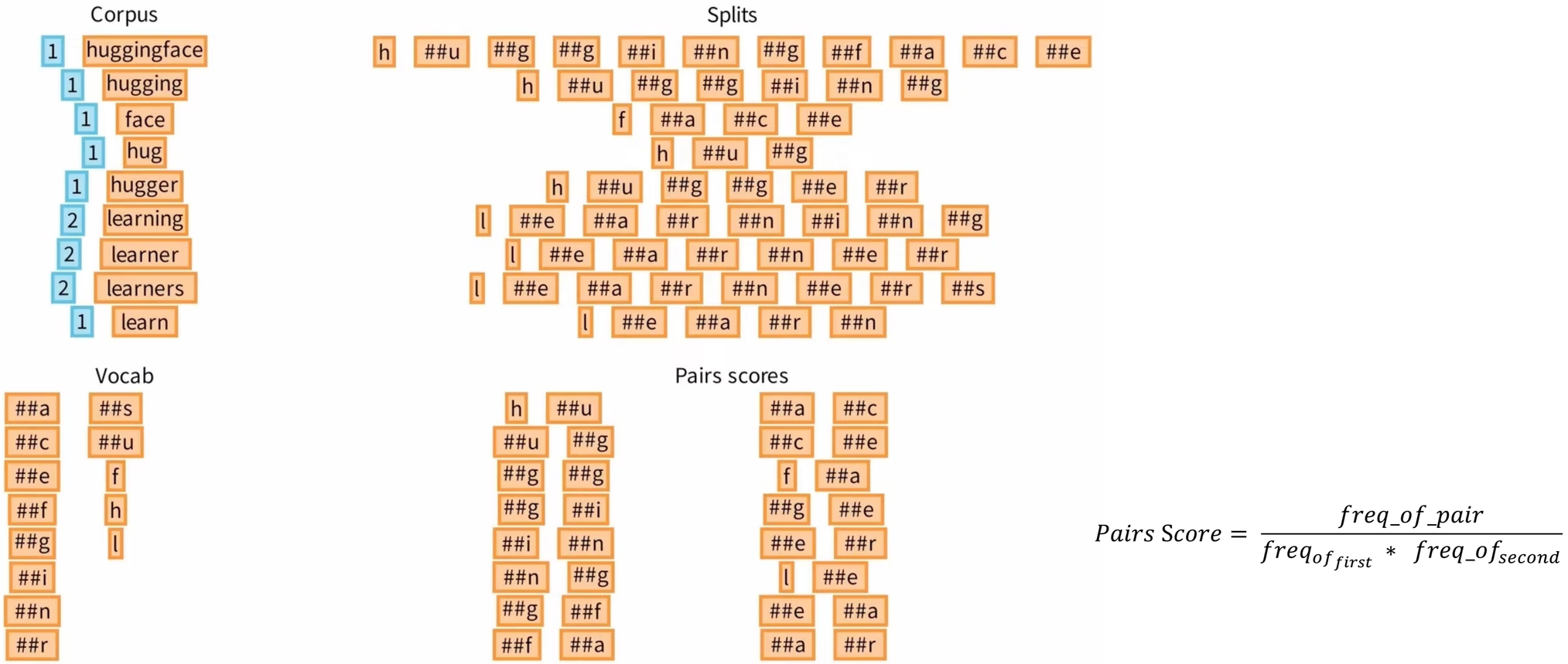
But why weed need to use a sub-word tokenizer??

https://huggingface.co/learn/nlp-course/chapter6/6?fw=pt

03 Pretraining BERT

- (1) a sub-word tokenizer : **WordPiece tokenizer**

  - Build a dictionary based on *Pairs Scores* until a certain number of sub-words



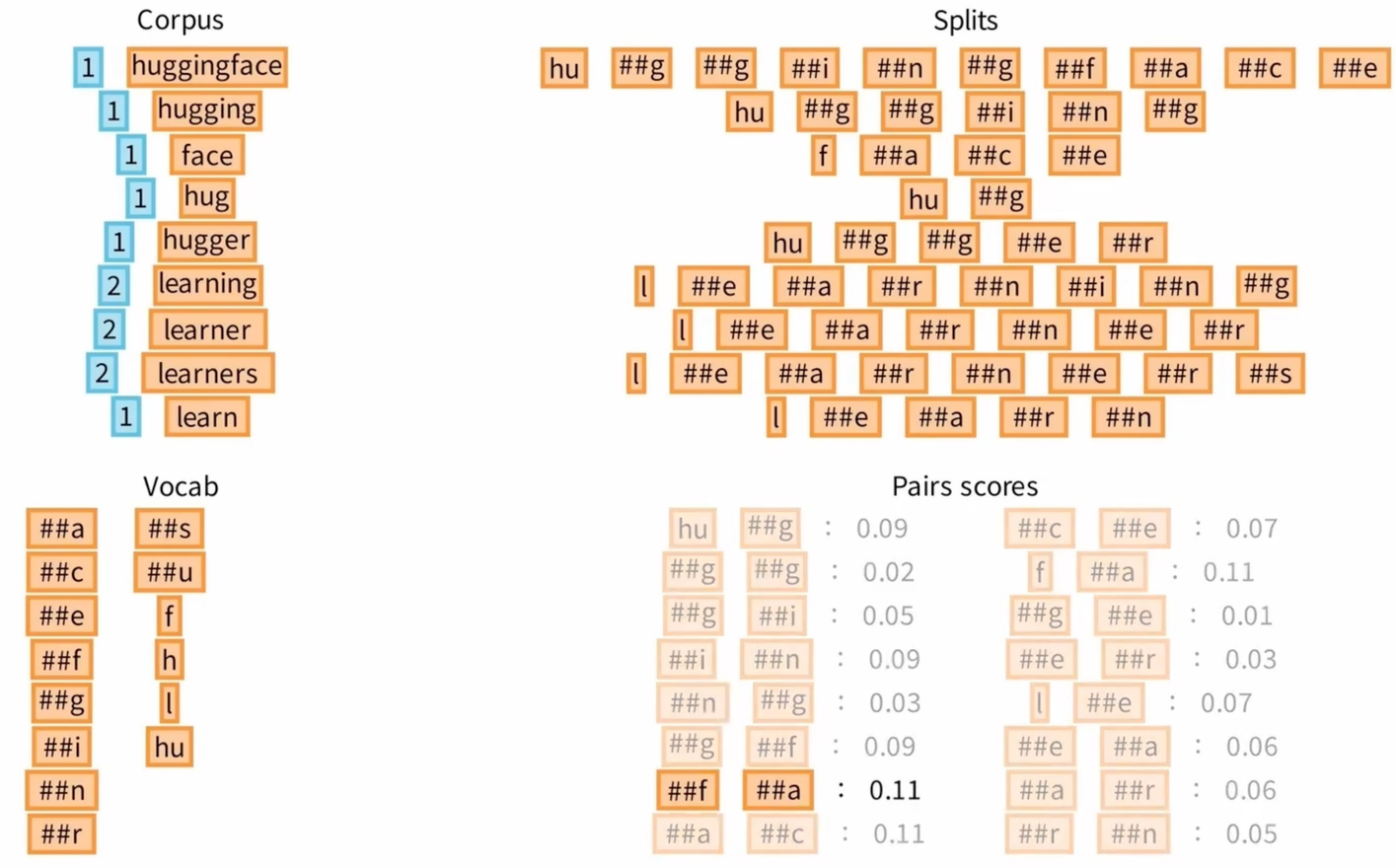
$$Pairs\ Score = \frac{freq\_of\_pair}{freq_{of\ first} * freq\_of_{second}}$$

https://huggingface.co/learn/nlp-course/chapter6/6?fw=pt

03 Pretraining BERT

- (1) a sub-word tokenizer : **WordPiece tokenizer**

  - Build a dictionary based on *Pairs Scores* until a certain number of sub-words



$$Pairs\ Score = \frac{freq\_of\_pair}{freq_{of\_first} * freq\_of_{second}}$$

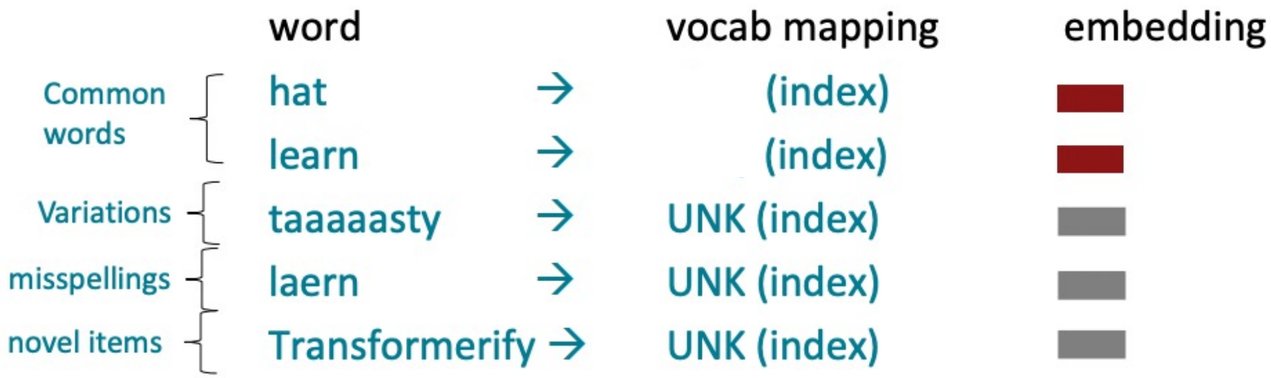
https://huggingface.co/learn/nlp-course/chapter6/6?fw=pt

03 Pretraining BERT

- **Why we need a sub-word tokenizer?**

Let's take a look at the assumptions we've made about a language's vocabulary.

We assume a fixed vocab of tens of thousands of words, built from the training set.
All *novel* words seen at test time are mapped to a single UNK.



Ref: cs224n from Stanford

03 Pretraining BERT

- **Why we need a sub-word tokenizer?**

Subword modeling in NLP encompasses a wide range of methods for reasoning about structure below the word level. (Parts of words, characters, bytes.)

- The dominant modern paradigm is to learn a vocabulary of **parts of words (subword tokens).**
- At training and testing time, each word is split into a sequence of known subwords.

Byte-pair encoding is a simple, effective strategy for defining a subword vocabulary.

1. Start with a vocabulary containing only characters and an "end-of-word" symbol.
2. Using a corpus of text, find the most common adjacent characters "a,b"; add "ab" as a subword.
3. Replace instances of the character pair with the new subword; repeat until desired vocab size.

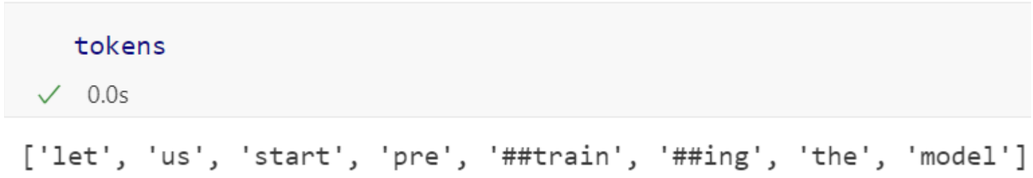
Originally used in NLP for machine translation; now a similar method (WordPiece) is used in pretrained models.

Ref: cs224n from Stanford

03 Pretraining BERT

BERT’s tokenizer: WordPiece Tokenizer

- *'let us start pretraining the model'* When this sentence is tokenized with WordPiece Tokenizer, the result is as follows:



- One word “*pretraining*” split into subwords like “pre”, “##train”, “##ing”

03 Pretraining BERT

- **BERT’s tokenizer: WordPiece Tokenizer**
- Because BERT can receive multiple sentences as input, it adds a [CLS] token at the beginning of the sentence and a [SEP] token at the end.

```
tokens
✓ 0.0s
['[CLS]', 'let', 'us', 'start', 'pre', '##train', '##ing', 'the', 'model', '[SEP]']
```

03 Pretraining BERT

- **BERT's tokenizer: WordPiece Tokenizer**
- For two sentences
  - Sentence A: "Paris is a beautiful city"
  - Sentence B: "I love Paris"

```
tokens
✓ 0.0s
['Paris', 'is', 'a', 'beautiful', 'city', 'I', 'love', 'Paris']
```

- Add a [CLS] token and a [SEP] token at the end of each sentence

```
tokens
✓ 0.0s
['[CLS]', 'Paris', 'is', 'a', 'beautiful', 'city', '[SEP]', 'I', 'love', 'Paris', '[SEP]']
```

03 Pretraining BERT

- From word embedding layer in BERT, it tokenizes the input sentence with

  (1) the WordPiece Tokenizer and then. ← we have done it!

  (2) transforms it into the three embeddings below, which are provided as input to BERT

  - Token embedding
  - Segment embedding
  - Position embedding

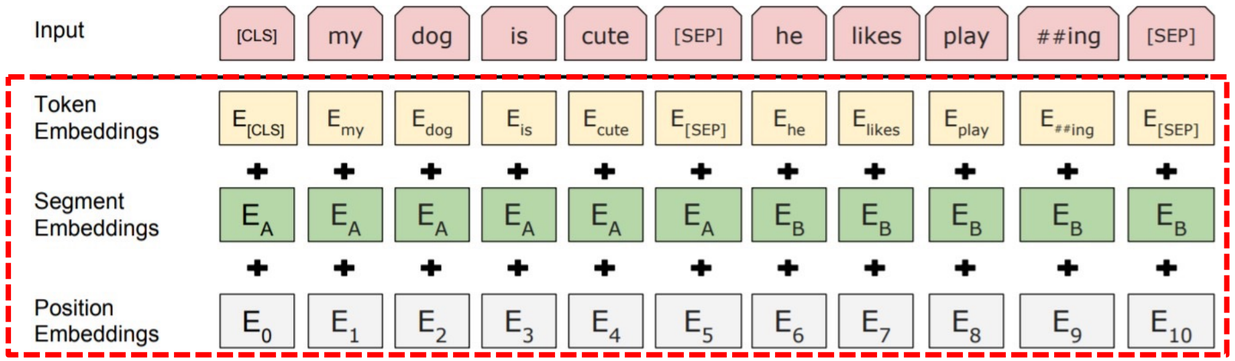
03 Pretraining BERT

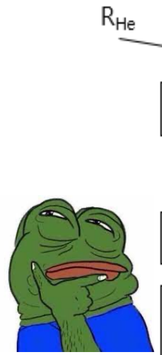
- From word embedding layer in BERT, it tokenizes the input sentence with

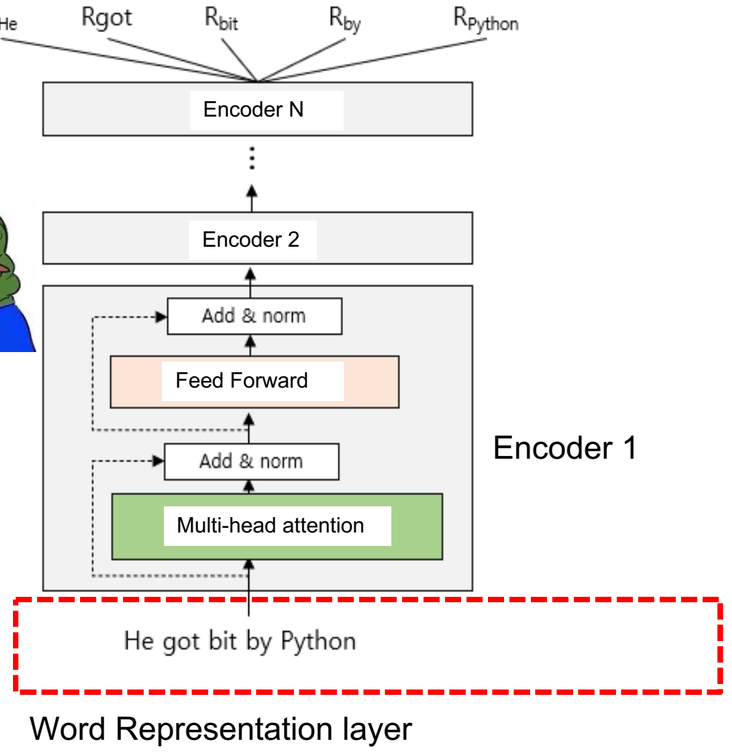
  (2) transforms it into the three embeddings below, which are provided as input to BERT

  - Token embedding
  - Segment embedding
  - Position embedding

Anyway, BERT needs embeddings from the input sentence



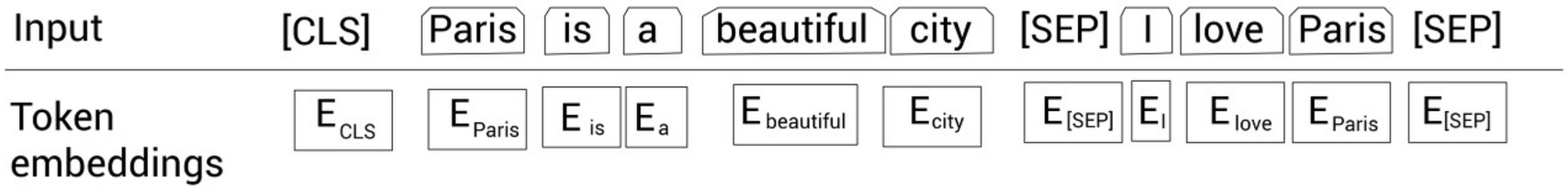




03 Pretraining BERT

1. BERT's word embedding layer

- **Token embedding:** Convert token to embedding

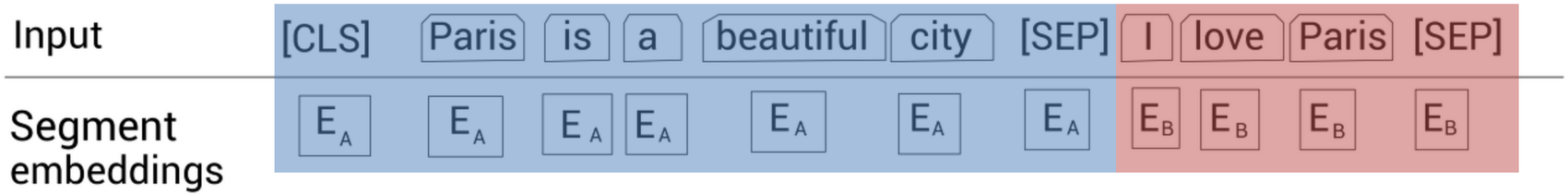


- Token embedding is trained in the pretraining step

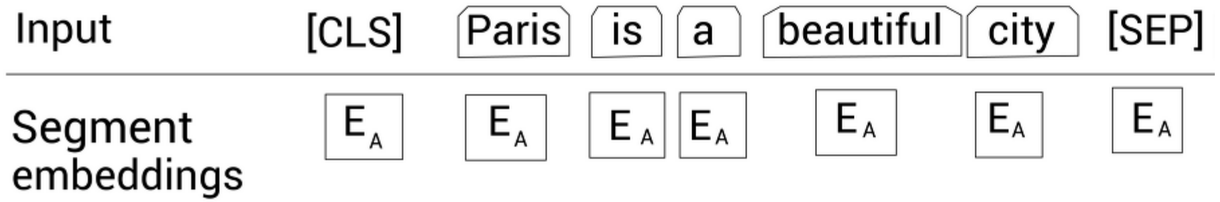
03 Pretraining BERT

1. BERT's word embedding layer

- **Segment embedding**: Used to distinguish between two given sentences
- The token corresponding to sentence A has the value E_A as its segment embedding, and the token corresponding to sentence B has the value E_B as its segment embedding.



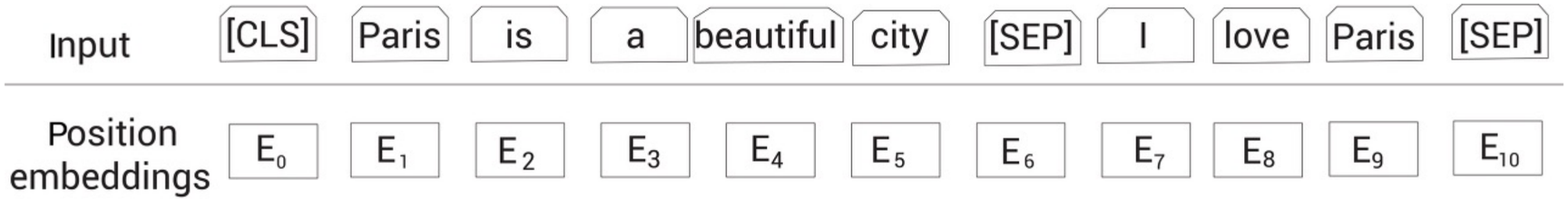
- If the input data is a single sentence, all tokens have the same segment embedding value



03 Pretraining BERT

1. BERT's word embedding layer

- **Position embedding**: Transformers process all words in parallel, so you need to provide information about the word order.

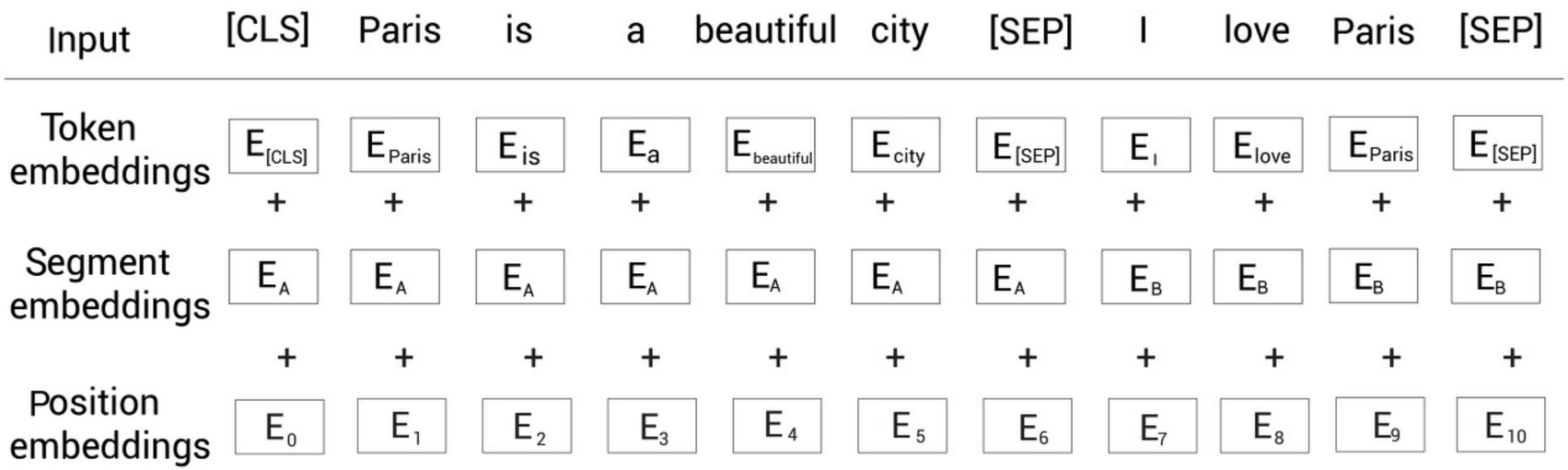


※ Unlike transformers, BERT uses Learned Positional Embeddings to provide positional information instead of using a sine wave function.

03 Pretraining BERT

1. BERT's word embedding layer

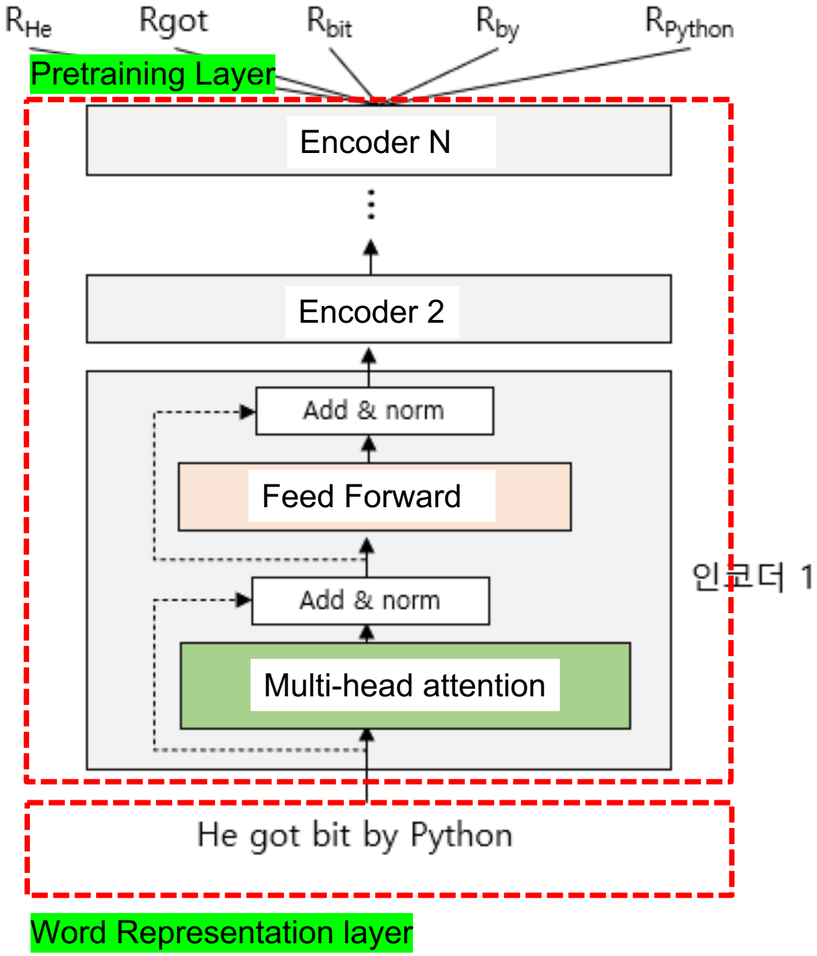
- **Final Word Embedding:** The final input representation for BERT takes a given sentence, tokenizes it with WordPress Tokenizer, finds three embedding values for each token, sums them up, and feeds them to BERT (Encoder of Transformer) as input.



03 Pretraining BERT

2. BERT's pretraining layer

- BERT uses the Toronto BookCorpus and English Wikipedia datasets for pretraining on the following two tasks:
  - MLM (Masked Language Modeling)
  - NSP (Next Sentence Prediction)

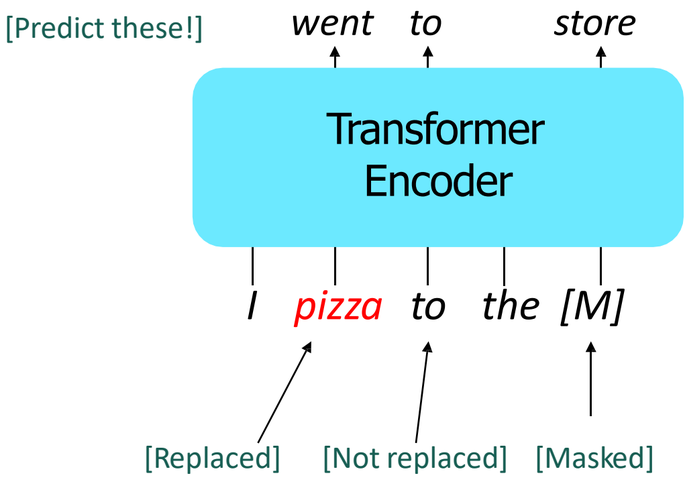


03 Pretraining BERT

2. BERT's pretraining layer

- **MLM, (Masked Language Modeling): Prediction of hidden words**

- Predict a random 15% of (sub)word tokens.
  - Replace input word with [MASK] 80% of the time
  - Replace input word with a random token 10% of the time
  - Leave input word unchanged 10% of the time (but still predict it!)
- Why? Doesn't let the model get complacent and not build strong representations of non-masked words. (No masks are seen at fine-tuning time!)



[Devlin et al., 2018]

03 Pretraining BERT

2. BERT's pretraining layer

- **MLM, (Masked Language Modeling): Prediction of hidden words**
- MLM randomly masks 15% of all words in a given input sentence and trains the model to predict the masked words.

```
tokens
✓ 0.0s
['[CLS]', 'Paris', 'is', 'a', 'beautiful', 'city', '[SEP]', 'I', 'love', 'Paris', '[SEP]']
```

- After tokenization as above, add [CLS] and [SEP] tokens and randomly mask 15% of the tokens.

```
tokens
✓ 0.0s
['[CLS]', 'paris', 'is', 'a', 'beautiful', '[MASK]', '[SEP]', 'i', 'love', 'paris', '[SEP]']
```

- 'city' token is masked and replaced with '[MASK]' token

03 Pretraining BERT

2. **BERT’s pretraining layer**

   MLM (Masked Language Modeling): Prediction of hidden words

- This approach leads to mismatches between pre-training and downstream tasks.
- For example, when you fine-tune a pre-trained BERT for a sentiment analysis task, the data entered for fine-tuning has discrepancies that do not have a [MAKS] token.

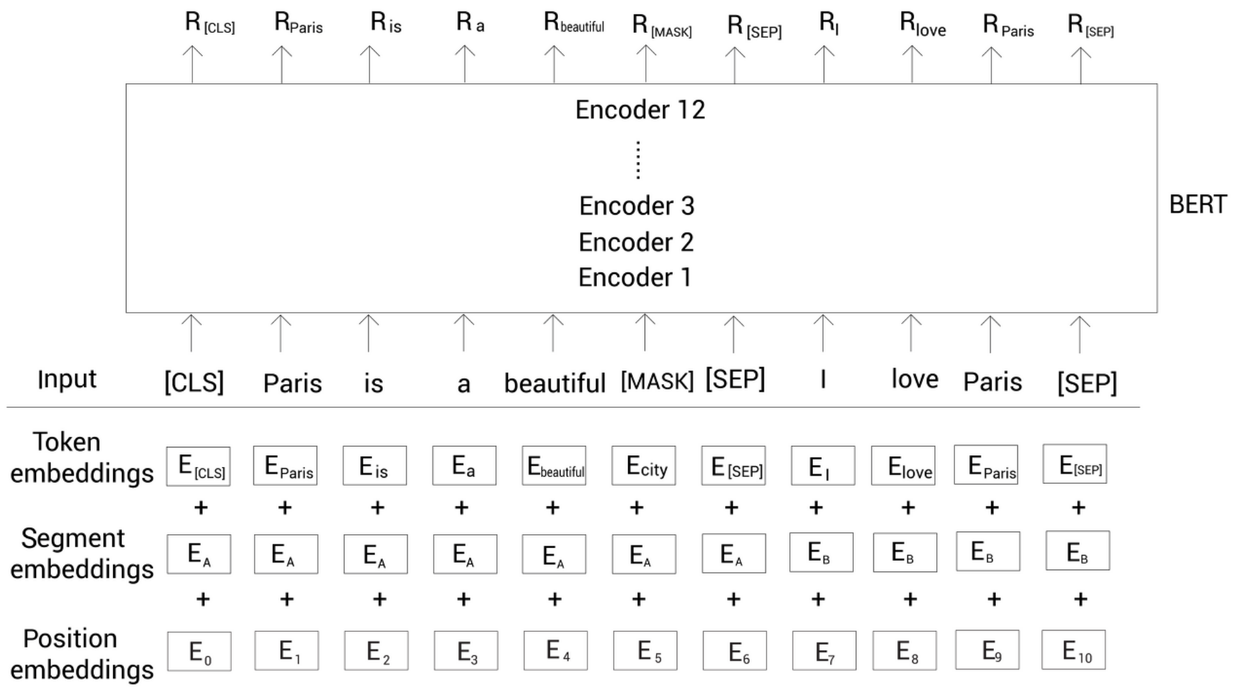
- To solve this problem, we introduced the 80-10-10% rule
  - Replace 80% of the 15% with [MASK] tokens
  - Replace 10% of the 15% with a random token (a random word)
  - Make no changes to 10% of the 15%.

03 Pretraining BERT

2. BERT's pretraining layer

MLM (Masked Language Modeling): Prediction of hidden words

- Once the masking is done, get the input embedding in the embedding layer and give it to BERT



03 Pretraining BERT

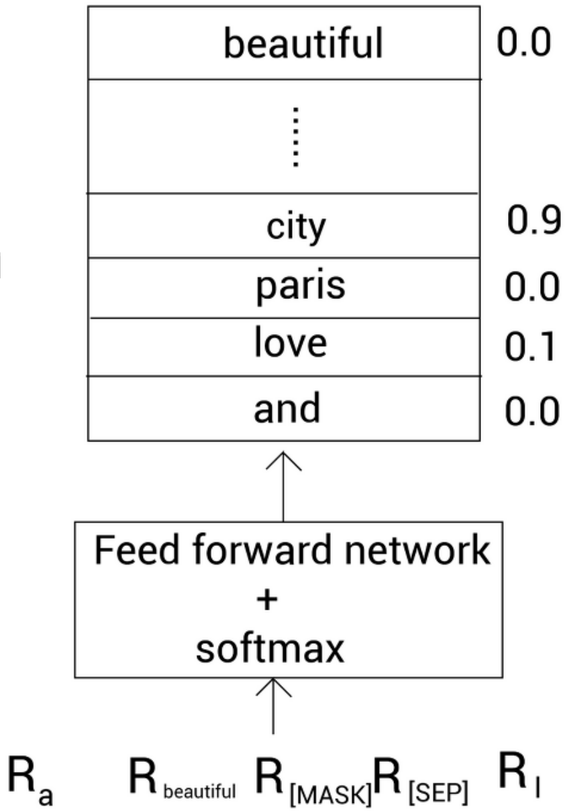
2. BERT's pretraining layer

MLM (Masked Language Modeling): Prediction of hidden words

- Once you have the representation of each token ($\boldsymbol{R}$), feed the representation of the masked token $\boldsymbol{R}_{([MASK])}$ into a feed-forward netwo (FFN) using the softmax function to predict the masked token.

- At the beginning of learning, the weights of FFN and encoder are not optimal, so the weights are updated by iterative learning through backpropagation to learn the optimal weights.

- MLM Task is also known as a cloze task



03 Pretraining BERT

2. BERT's pretraining layer

NSP (Next Sentence Prediction)

- Given sentences A and B, a binary classification task to predict whether B is the next sentence after A.

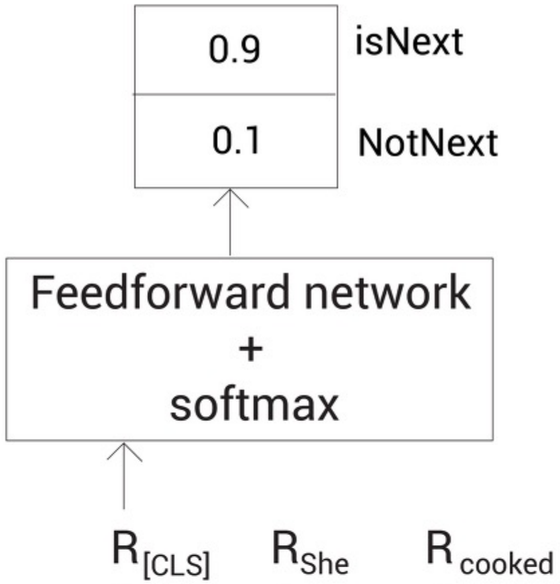
Sentence pairs	label
She cooked pasta It was delicious	IsNext
Jack loves songwriting He wrote a new song	isNext
Birds fly in the sky He was reading	NotNext
Turn the radio on She bought a new hat	NotNtext

03 Pretraining BERT

2. BERT's pretraining layer

NSP (Next Sentence Prediction)

- Obtain the embedding of sentences A, B.
- Feed the representation of the [CLS] token into FFN, softmax for binary classification



Why use only [CLS] tokens?

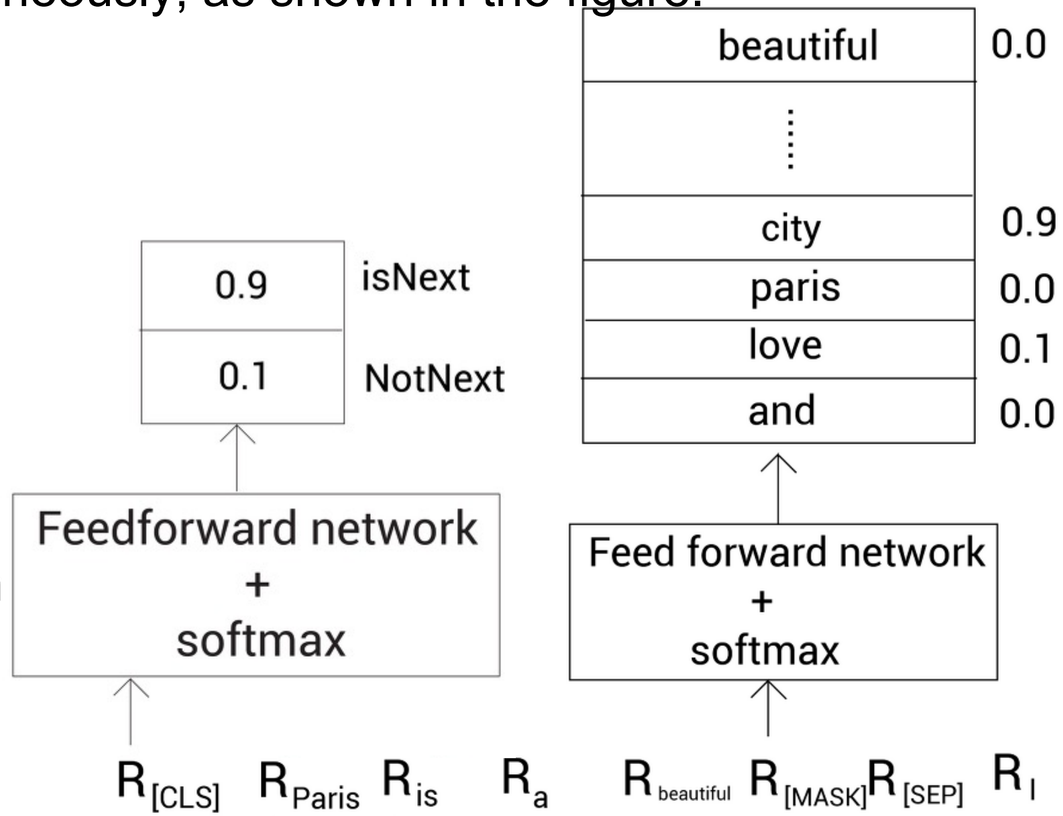
※ The [CLS] token holds the aggregate representation of all tokens by default, so it holds the representation of the entire sentence

03 Pretraining BERT

2. BERT's pretraining layer

NSP (Next Sentence Prediction)

- Pretrain BERT using MLM and NSP tasks simultaneously, as shown in the figure.
- Batch size : 256, 1,000,000 steps, Adam optimizer, lr : 1e-4, β1 : 0.9, β2 : 0.999
- Use a warm-up step with a high learning rate in the beginning and a low learning rate in the end
- Apply dropout to all layers with a dropout probability of 0.1 and use GELU for the activation function



Thank you